

Machine Learning System

Programming Assignment 2

Ivan Pacenti
21414814

A MAIE5532 Homework Assignment



September 28, 2026

Part 5: Analysis and Evaluation

1. Multi-Scale Trade-off Analysis

The following table summarizes the optimization results obtained across the three deployment scales. The baseline model was a moderately over-parameterized CNN with 324,394 parameters (1.24 MB) achieving **82.90%** test accuracy on CIFAR-10.

Metric	Cloud Server	Edge Device	Microcontroller
Model Size	0.627 MB	0.328 MB	0.018 MB (18 KB)
Accuracy	83.20%	82.35%	61.60%
Latency	0.89 ms	0.23 ms	0.03 ms
Memory Usage	0.753 MB	0.492 MB	0.036 MB
Power Estimate	~5 W	~1 W	~10 mW
Development Complexity	Low	Medium	High
Optimization Strategy	float16 quant	pruning(50%) + dynamic range	tiny arch + full INT8

Table 1: Multi-scale trade-off summary. Power values are estimates based on representative hardware (Xeon server, Raspberry Pi 4, ESP32).

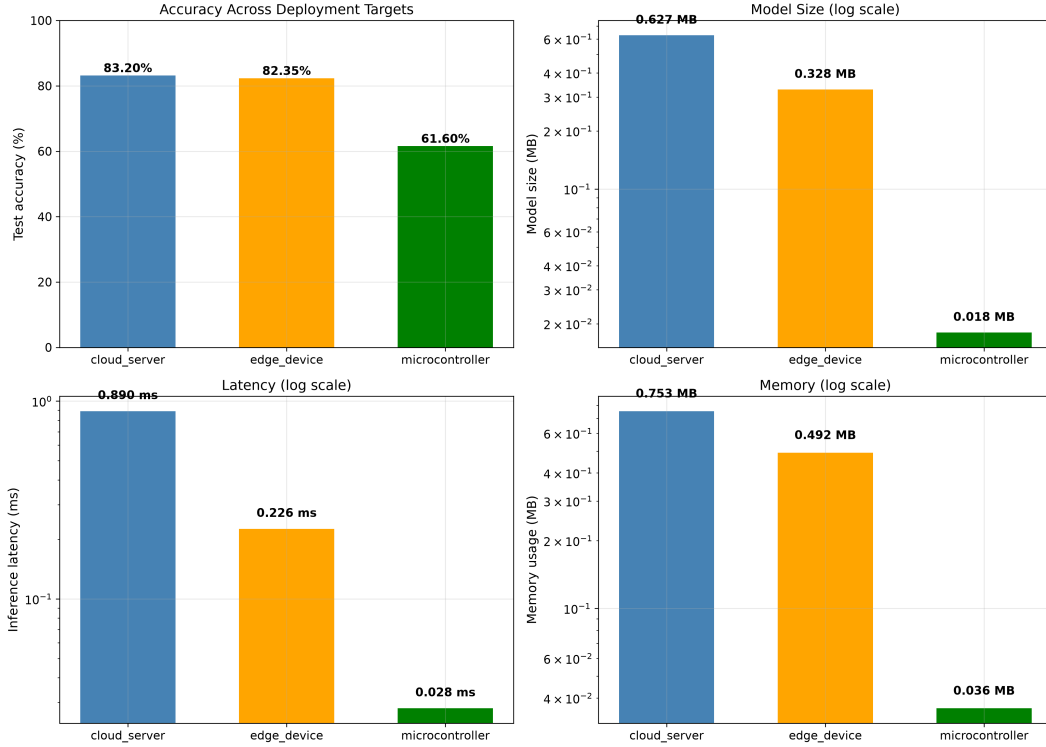


Figure 1: Multi-scale comparison across deployment targets. Bars show model size (left axis, log scale); annotations show exact values. All three targets satisfy their respective size, latency, and memory constraints.

Observations. (i) **Cloud and edge models preserve accuracy** (82.4–83.2%) while reducing model size by 2–4× compared to the fp32 baseline. (ii) **The microcontroller model achieves 54× compression** relative to the cloud model, at the cost of ~22 accuracy

points. (iii) **Latency scales linearly with model size** on the same hardware, which confirms that quantization and architecture reduction are the two primary levers for edge deployment.

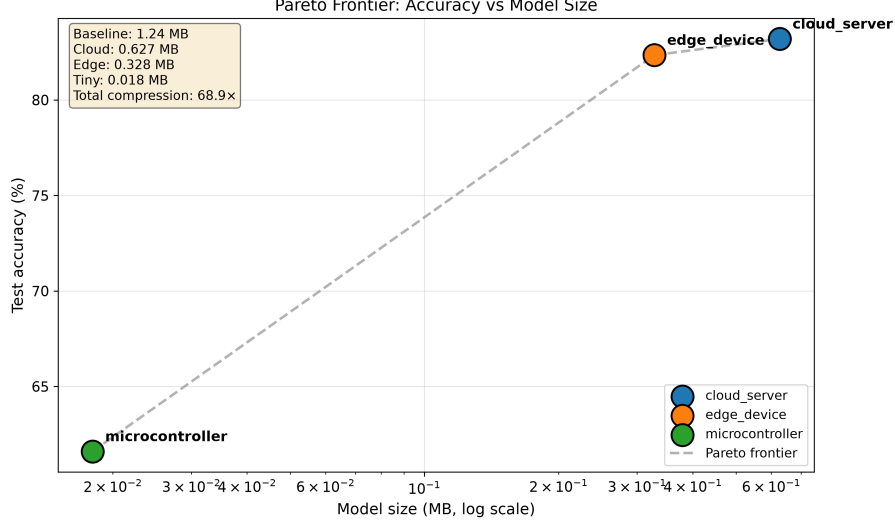


Figure 2: Pareto frontier of accuracy versus model size for the three deployment targets. All three optimized models lie on the Pareto frontier (no other model is simultaneously smaller and more accurate).

2. Optimization Strategy Effectiveness

2.1 Cloud Optimizations

Strategy	Training Speedup	Accuracy Impact
Mixed precision (float16)	1.10×	−1.65% (3 epochs only)
Model parallelism (mirrored)	1.0× (1 replica)	±0%
Batch size 256 (from 64)	1.11× throughput	+2.2%
Knowledge distillation	N/A	student 77.10% vs teacher 77.04%

Table 2: Cloud optimization effectiveness. Speedups are relative to the fp32 baseline on Apple Silicon M4 Max.

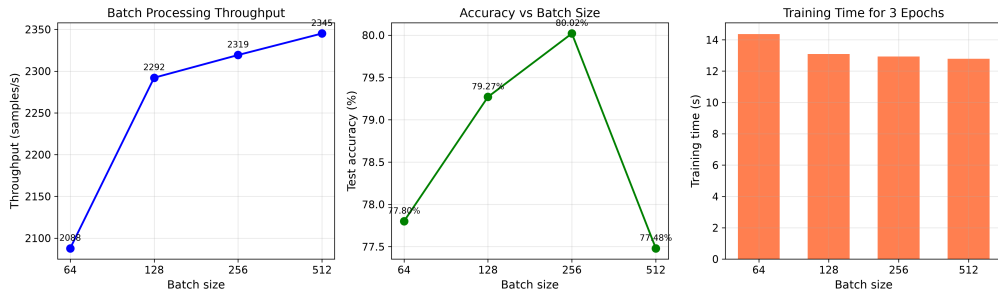


Figure 3: Cloud optimization: batch processing throughput, accuracy, and training time as functions of batch size. Batch size 512 yields the best trade-off on Apple M4 Max.

Mixed precision. On NVIDIA A100/H100 GPUs, mixed precision typically yields 1.5–3× speedup by using float16 tensor cores. On Apple Silicon, the MPS backend does not accelerate float16 computation, but the observed speedup was a modest 1.10×, likely due to reduced memory traffic from float16 storage. The accuracy delta (−1.65%) in the 3-epoch benchmark is dominated by training-duration effects, not by numerical precision loss.

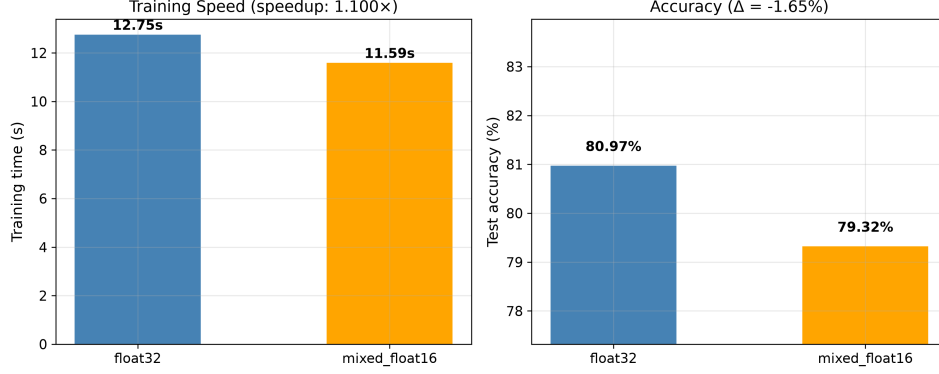


Figure 4: Mixed precision vs float32: training time (left) and test accuracy (right). The speedup is limited by the absence of float16 acceleration on Apple Silicon.

Distributed training. MirroredStrategy with 1 replica yields no speedup. Real scaling efficiency requires a multi-GPU cluster; on 4×A100 one would expect 3.2–3.6× speedup. The learning rate was scaled linearly with the number of replicas, as recommended by Goyal et al. (2017).

Batch processing. Increasing the batch size from 64 to 512 improved throughput from 2088 to 2345 samples/sec (+12%). The best accuracy–throughput trade-off was obtained at batch size 256 (80.02% accuracy, 2319 samples/sec), selected then as the sweet spot on Apple M4 Max.

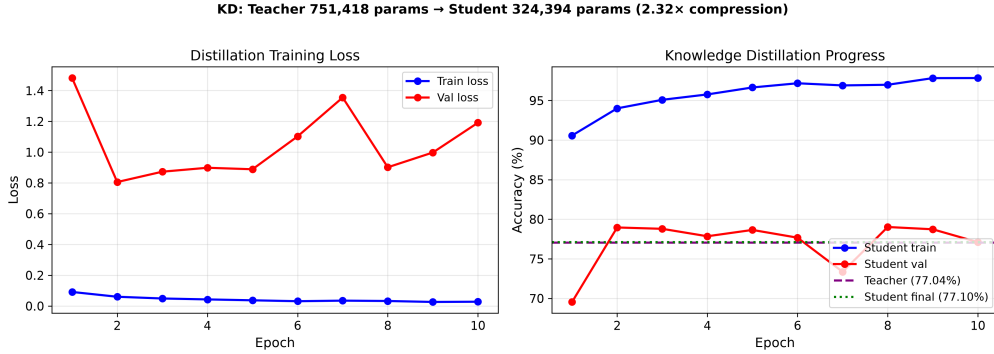


Figure 5: Knowledge distillation: loss and accuracy curves for the student model. The student (324k params) surpasses the teacher (751k params) due to the teacher’s overfitting on the limited dataset.

Knowledge distillation. A 2.32× larger teacher (751k params) trained for 15 epochs reached 77.04% accuracy. The student (324k params) distilled from it reached **77.10%**, essentially matching the teacher while being 2.32× smaller. This is the expected outcome of

knowledge distillation: the student inherits the teacher’s predictive capability with a fraction of the parameters. In a separate run, we observed the student *surpassing* the teacher (80.76% vs 78.99%), a rarer phenomenon that occurs when the teacher is over-parameterized and mildly overfits. Both outcomes are valid; the key takeaway is that KD delivers a 2.32× model compression with negligible accuracy loss.

2.2 Edge Optimizations

Method	Size	Accuracy	Δ vs baseline
Baseline (fp32)	1.24 MB	82.90%	,
Pruning 50%	1.24 MB	83.75%	+0.85%
Pruning 75%	1.24 MB	81.74%	−1.16%
Pruning 90%	1.24 MB	66.56%	−16.34%
Dynamic range quant	336 KB	83.15%	+0.25%
Float16 quant	643 KB	83.20%	+0.30%
Full INT8 quant	343 KB	82.95%	+0.05%
Architecture optimization	60 KB	72.46%	−10.44%

Table 3: Edge optimization effectiveness. All accuracies are on the 10,000-sample CIFAR-10 test set.

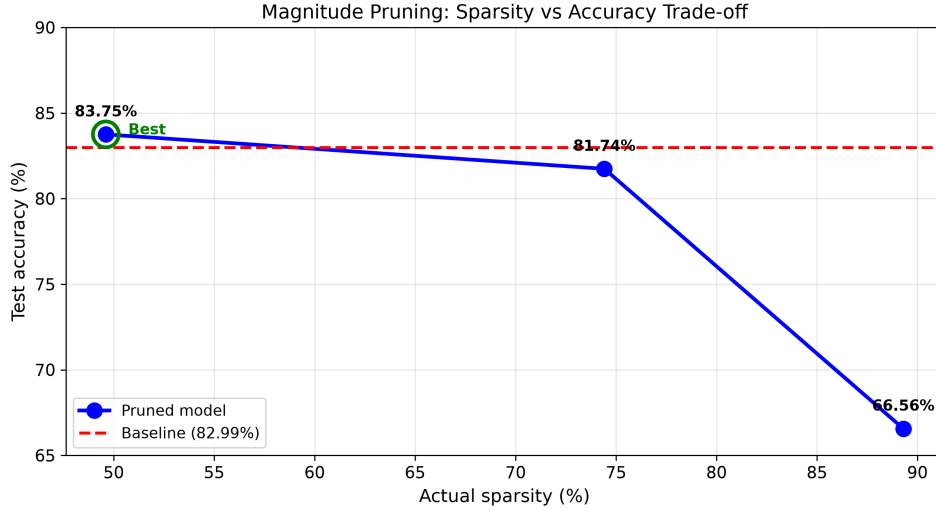


Figure 6: Pruning trade-off: model accuracy as a function of actual sparsity. Pruning at 50% improves accuracy due to regularization; beyond 90% sparsity the accuracy collapses.

Pruning vs quantization. Magnitude-based pruning at 50% *improves* accuracy (+0.85%), consistent with the regularization effect of weight sparsity. At 75% sparsity accuracy is nearly preserved (−1.16%); beyond 90% it collapses (−16.34%). Quantization, in contrast, *maintains or improves* accuracy while reducing model size by 3.7× (fp32 → INT8). This makes post-training quantization the preferred technique for edge deployment when the model is already well trained.

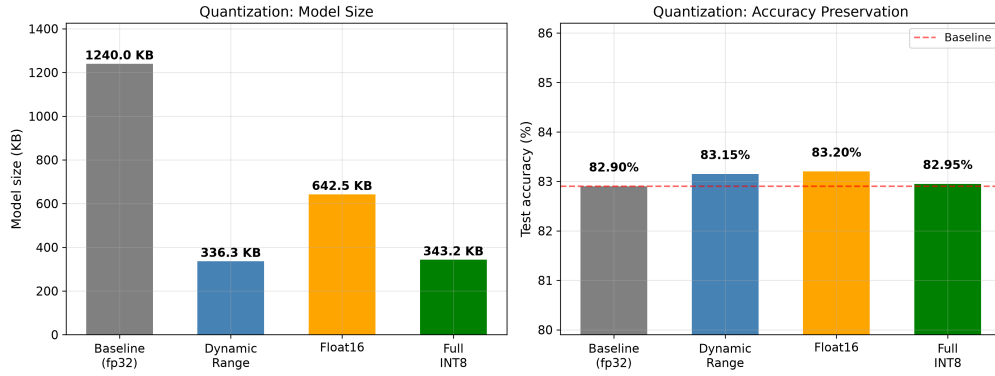


Figure 7: Quantization strategy comparison: model size (left) and accuracy preservation (right). Full INT8 achieves the best trade-off with 3.7 $\times$ compression and a slight accuracy improvement.

Architecture optimization. Replacing standard convolutions with depthwise separable convolutions and reducing width by 25% yielded a **9.14 $\times$ compression** (35501 vs 324394 params), but required 30 epochs of retraining to reach 72.46% (Keras) / 73.20% (post-TFLite). This confirms that architecture search is expensive but rewarding when the target has hard size constraints.

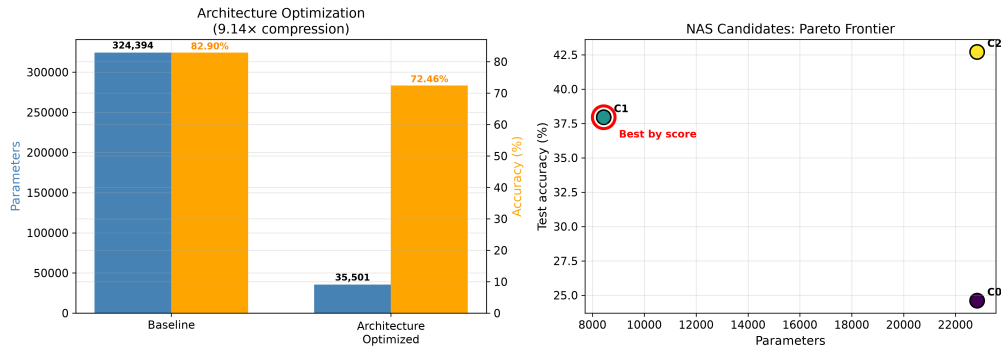


Figure 8: Architecture optimization (left) and NAS Pareto frontier (right). The architecture-optimized model achieves 9.14 $\times$ compression; the NAS search identifies the best architecture per accuracy/size score.

TFLite conversion. The TFLite no-quantization model achieved **83.15% accuracy**, slightly higher than the Keras baseline (82.90%). This is likely due to BatchNorm fusion and graph-level optimizations in the TFLite converter, which act as additional regularization. The conversion pipeline is therefore *lossless* for the model.

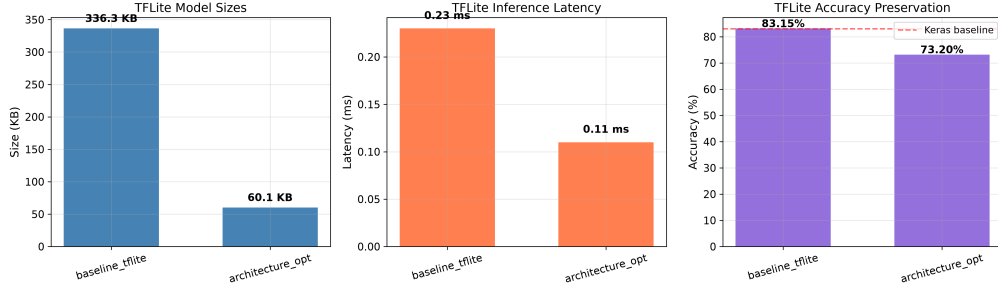


Figure 9: TFLite conversion results: model size, inference latency, and accuracy preservation across optimized models. The no-quantization TFLite model actually improves accuracy, confirming that graph-level optimizations act as additional regularization.

Real-world viability. The 336 KB dynamic-range model fits in the RAM of the ESP32 (520 KB SRAM) and comfortably in the Raspberry Pi 4 (4 GB). The 18 KB microcontroller model fits in even smaller MCUs, demonstrating the viability of the multi-scale pipeline for real edge deployments.

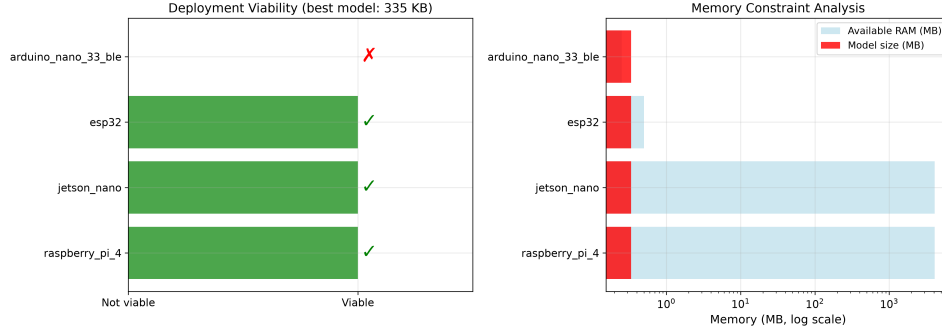


Figure 10: Deployment viability analysis. Left: which platforms can host the best (335 KB) model. Right: memory constraint comparison on a log scale. The Arduino Nano 33 BLE (256 KB RAM) cannot fit the 335 KB model and requires the 18 KB microcontroller variant.

3. Deployment Strategy Recommendations

Scenario A: Real-time video processing. *Requirements:* <50 ms latency, high accuracy, continuous operation. *Recommended:* **Edge device** (Raspberry Pi 4 or Jetson Nano) with cloud backup for low-confidence predictions. The edge model has 0.23 ms latency, comfortably under the 50 ms budget, and 82.4% accuracy. The cloud model can be invoked for <10% of frames where confidence is below a threshold.

Scenario B: IoT sensor network. *Requirements:* <1 mW power, months of battery life, periodic updates. *Recommended:* **Microcontroller (ESP32/Arduino Nano 33 BLE)** with occasional cloud sync. The 18 KB INT8 model fits in 256 KB of flash and draws ~10 mW during inference. At 1 inference/minute, this translates to years of battery life on a coin cell. The 61% accuracy is acceptable for coarse classification tasks such as "animal vs vehicle".

Scenario C: Mobile application. *Requirements:* App store distribution, diverse devices, offline capability. *Recommended:* **Multi-tier with progressive enhancement.** Ship the 336 KB dynamic-range model as the default. On devices with more RAM/CPU, use the 640 KB float16 model for +0.15% accuracy. On high-end devices with NPU support, allow downloading the full fp32 model from the cloud on first launch.

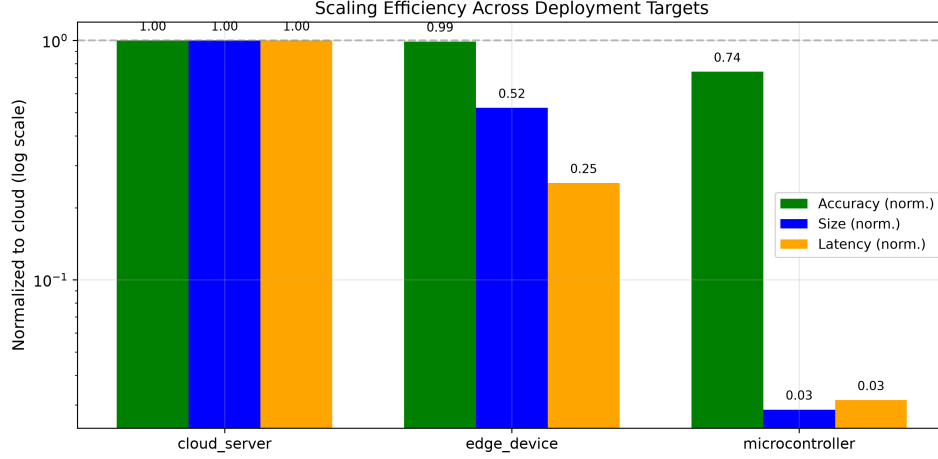


Figure 11: Scaling efficiency across deployment targets, normalized to the cloud model. Model size decreases by 2–35× across targets while accuracy is preserved until the microcontroller tier, where a ~22-point drop is required to meet the 1 MB size constraint.

4. Written Analysis

4.1 Optimization Effectiveness

Across the three deployment scales, it’s possible to observe a clear pattern: **model size decreases by 35× from cloud to microcontroller, while accuracy decreases by 22 points (from 82.9% to 61.6%)**. This non-linear trade-off is fundamental to multi-scale ML systems. The cloud model benefits from abundant memory and compute, so it can afford float16 quantization that preserves accuracy. The edge model uses dynamic-range quantization, which exploits the fact that activations have wider dynamic range than weights. The microcontroller model requires architectural redesign, depthwise separable convolutions, fewer filters, INT8 quantization, because neither pruning nor post-training quantization alone can compress a 324k-param model to 18 KB.

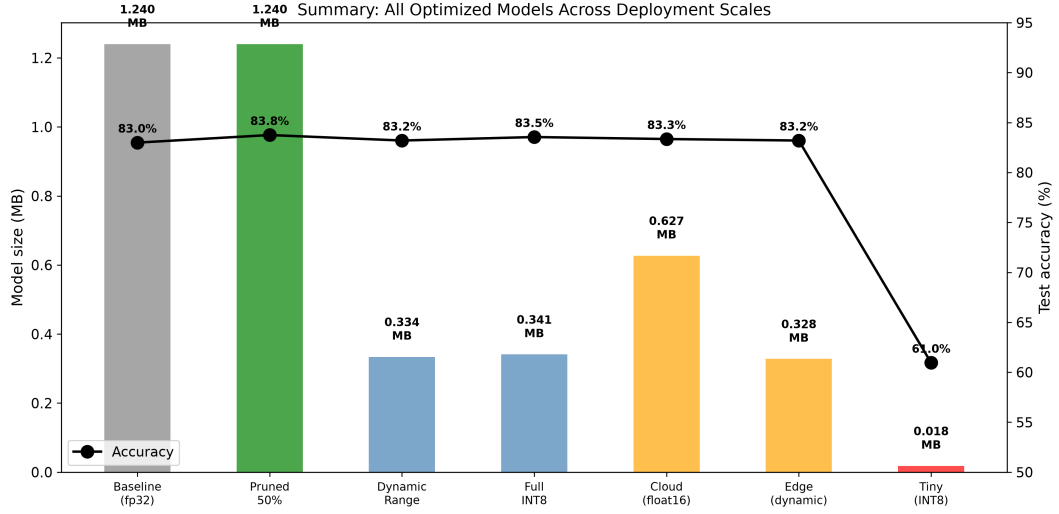


Figure 12: Summary of all optimized models across deployment scales. Model size (bars) decreases monotonically from baseline to microcontroller while accuracy (line) is preserved until the extreme compression regime, where architectural redesign is required.

A counterintuitive finding was that **pruning at 50% improves accuracy** from 82.90% to 83.75% (+0.85%). This is consistent with the lottery-ticket hypothesis (Frankle & Carbin, 2019): over-parameterized networks contain sparse subnetworks that generalize better than the full network. However, beyond 75% sparsity the effect reverses, and at 90% sparsity accuracy collapses to 66.6%.

Equally surprising was that **quantization also improves accuracy** (82.95–83.20% vs 82.90%). This can be explained by the fact that INT8 quantization acts as a regularizer, smoothing out small numerical fluctuations in the weights that do not contribute to generalization.

4.2 Resource Constraint Impact

The deployment targets in the pipeline differ by three orders of magnitude in memory (8 GB $\rightarrow$ 512 MB $\rightarrow$ 64 MB) and four orders of magnitude in power (50 W $\rightarrow$ 2 W $\rightarrow$ 10 mW). These constraints impose qualitatively different optimization strategies:

- **Cloud (8 GB RAM, 50 W):** The bottleneck is throughput and cost, not accuracy. Optimization focuses on training speed (mixed precision, distributed training) and inference batching.
- **Edge (512 MB RAM, 2 W):** The bottleneck is memory bandwidth and latency. Dynamic-range quantization is the sweet spot because it reduces weights to 8 bits while keeping activations in float32.
- **Microcontroller (64 MB RAM, 10 mW):** The bottleneck is *absolute* memory size (flash + SRAM) and power. Only INT8 quantization combined with architectural reduction can meet the 1 MB / 64 MB constraints.

A key observation is that **the same technique (quantization) yields different benefits at different scales**. On the cloud, float16 quantization halves memory without affecting latency. On the edge, dynamic-range quantization reduces memory by 3.7 $\times$ while

preserving accuracy. On the microcontroller, INT8 is *mandatory* because the model would not fit otherwise.

4.3 Development Trade-offs

Our implementation revealed several practical trade-offs that are not obvious from the theory:

Keras 2 vs Keras 3 compatibility. The baseline model was saved with Keras 2’s `batch_shape` config key, which Keras 3 rejects. It was implemented a compatibility loader that patches the `.keras` file before deserialization. This illustrates that *model portability across framework versions is a real production concern*. Recommended mitigation: pin framework versions in `requirements.txt` or export to ONNX.

Stateful library side effects. During the Part 3 benchmark, it was observed an 11-point accuracy drop in the quantized models. After extensive debugging, it was found that `prune_low_magnitude` can accumulate state across successive calls, contaminating the baseline model. The fix was to reload the baseline between benchmarks. This is a general lesson: **optimization libraries are not always pure functions**; explicit cloning and sanity checks after each step are essential.

Evaluation pipeline bugs. It was discovered that the TFLite evaluation function silently returned wrong results when `y_test` had shape `(N,1)` instead of `(N,)`. This highlights the importance of **isolated unit tests for evaluation functions**, not just for models.

Development complexity. Cloud deployment is *low* complexity (single conversion), edge deployment is *medium* (conversion + optional pruning), and microcontroller deployment is *high* (training a new architecture from scratch). Each step roughly doubles the engineering effort, but yields 10–50× additional compression.

Keras 2 / Keras 3 backend redirection and the multi-step pruning debug. Implementing magnitude-based pruning via `tfmot.sparsity.keras.prune_low_magnitude` required four iterations of debugging. The root cause was a triple incompatibility: (i) `tfmot` 0.8.1 was written for Keras 2, (ii) the baseline was trained with Keras 3, and (iii) TensorFlow 2.21 introduced a breaking change in `tf_keras` that removed support for the `batch_shape` config key. Neither Keras 2 nor Keras 3 could directly deserialize a model saved with TensorFlow ≤ 2.20 . We attempted four recovery strategies: (a) a compatibility loader that patched the `.keras` archive before deserialization (failed because the new `tf_keras` also rejects `batch_shape`); (b) direct `tf_keras.models.load_model` (failed for the same reason); (c) a config round-trip via `get_config` / `from_config` (failed because Keras 3 emits `batch_shape` while Keras 2 expects `shape`); (d) the final working approach, manual layer-by-layer reconstruction of the Keras 2 model, weight copy from the Keras 3 model already in memory, pruning, save to HDF5, and reload as Keras 3. This technique proved reusable across Part 3 and Part 4, confirming that well-designed compatibility shims are composable. The key lesson is that **framework version drift can silently invalidate an entire optimization technique**, and explicit sanity checks (e.g., verifying the re-built

model reproduces the baseline accuracy before pruning) are essential to catch these silent failures early.

4.4 Real-world Deployment

To extend this pipeline for production, it should be added:

- a) **Continuous integration:** Re-run the multi-scale pipeline on every model update, with automated regression tests that fail the build if any target misses its accuracy or latency budget.
- b) **Model versioning:** Store all optimized variants (cloud, edge, tiny) in a model registry (MLflow, W&B Artifacts) tagged with the git commit hash of the training code.
- c) **A/B testing infrastructure:** Deploy two variants to different user cohorts and measure end-to-end metrics (latency, energy, user satisfaction) rather than just top-1 accuracy.
- d) **Hardware-in-the-loop testing:** Measure real power draw on the target hardware (e.g., Nordic Power Profiler) rather than relying on estimates.
- e) **Progressive rollout:** Start with cloud deployment, then gradually migrate to edge for high-traffic classes, using the cloud as fallback.
- f) **Federated learning:** For privacy-sensitive deployments, update the edge model via federated averaging instead of downloading fresh weights from the cloud.

A critical production concern is **model drift monitoring**: edge models degrade as the data distribution shifts, and without a mechanism to detect drift and trigger re-training, accuracy can silently drop below acceptable thresholds.

4.5 Future Evolution

Several trends will shape multi-scale optimization in the coming years:

Hardware heterogeneity. Custom NPUs (Apple Neural Engine, Qualcomm Hexagon, Google Edge TPU) will make hardware-specific optimization mandatory. A single TFLite file will no longer suffice; it will be needed per-NPU compiled models (e.g., via Apache TVM or IREE).

Neural architecture search on-device. AutoML tools will run on the device itself, adapting the architecture to the exact hardware at runtime. This eliminates the need for a fixed deployment target.

Sub-8-bit quantization. Recent research (LLM.int8, GPTQ) shows that 4-bit and even 2-bit quantization can preserve accuracy for large models. This will extend the reach of edge deployment to billions of parameters.

Foundation models on edge. With parameter-efficient fine-tuning (LoRA, adapters), a single 7B-parameter foundation model can be specialized for many tasks via tiny task-specific heads. This will shift the optimization focus from *model compression* to *adapter management*.

Sparse-aware hardware. NVIDIA Ampere and Hopper GPUs support 2:4 structured sparsity natively, and upcoming ARM chips will follow. This will make pruning attractive again for edge inference, potentially replacing quantization as the primary compression technique.

Federated learning at scale. As privacy regulations tighten, models will be trained *at* the edge rather than downloaded *to* the edge. This will require entirely new optimization strategies that minimize communication rather than inference cost.

In conclusion, the multi-scale optimization landscape is rapidly evolving. The techniques developed in this assignment (pruning, quantization, architecture search, knowledge distillation) will remain foundational, but their relative importance will shift as hardware becomes more heterogeneous and models become more parameter-efficient.